Actividad Individual
Código de experimentación base con MLflow

Gerardo González Martínez, A01840096, 11 de octubre de 2026

https://github.com/gerardoglezmart/mna-tc5061-actividad3-experimentacion-mlflow

---

1. Migración del notebook a un script

Carga, preprocesamiento, entrenamiento y evaluación pasaron del notebook de la semana 3 a funciones con una sola responsabilidad, en un proyecto generado con Cookiecutter Data Science (`ccds`). `main.py` las ejecuta en orden (Código 1) y el notebook queda solo para la exploración de datos.

Etapa	Archivo	Funciones
Carga de datos	`wine_quality/dataset.py`	`cargar_datos`
Preprocesamiento	`wine_quality/dataset.py`	`limpiar_datos`, `dividir_datos`
Entrenamiento	`wine_quality/modeling/train.py`	`construir_modelo`
Evaluación	`wine_quality/modeling/evaluate.py`	`evaluar`, `guardar_artefactos`
Ejecución y registro	`main.py`	`leer_argumentos`, `main`

Código 1: `main.py`: orden de ejecución con las funciones de cada etapa.

```
def main() -> None:
    args = leer_argumentos()

    # Mismo orden del notebook de la semana 3: datos, modelo, evaluación
    X_train, X_test, y_train, y_test = dividir_datos(limpiar_datos(cargar_datos()), args.seed)
    modelo = construir_modelo(args.C, args.class_weight, args.max_iter, args.seed)
    modelo.fit(X_train, y_train)
    pred = modelo.predict(X_test)
    metricas = evaluar(y_test, pred, modelo.predict_proba(X_test)[:, 1])
```

2. Parametrización de la ejecución

Tres hiperparámetros y la semilla se exponen con `argparse`, con valor por defecto y ayuda descriptiva. Sin argumentos se reproduce el baseline de la semana 3 (Figura 1).

Argumento	Default	Descripción
`--C`	1.0	Inverso de la fuerza de regularización
`--class-weight`	none	`balanced` da más peso a la clase minoritaria
`--max-iter`	1000	Máximo de iteraciones del optimizador lbfgs
`--seed`	42	Semilla de la partición train/test

Comando exacto, desde la raíz del repositorio y con el ambiente activado:

```
python main.py --class-weight balanced --C 1.0
```

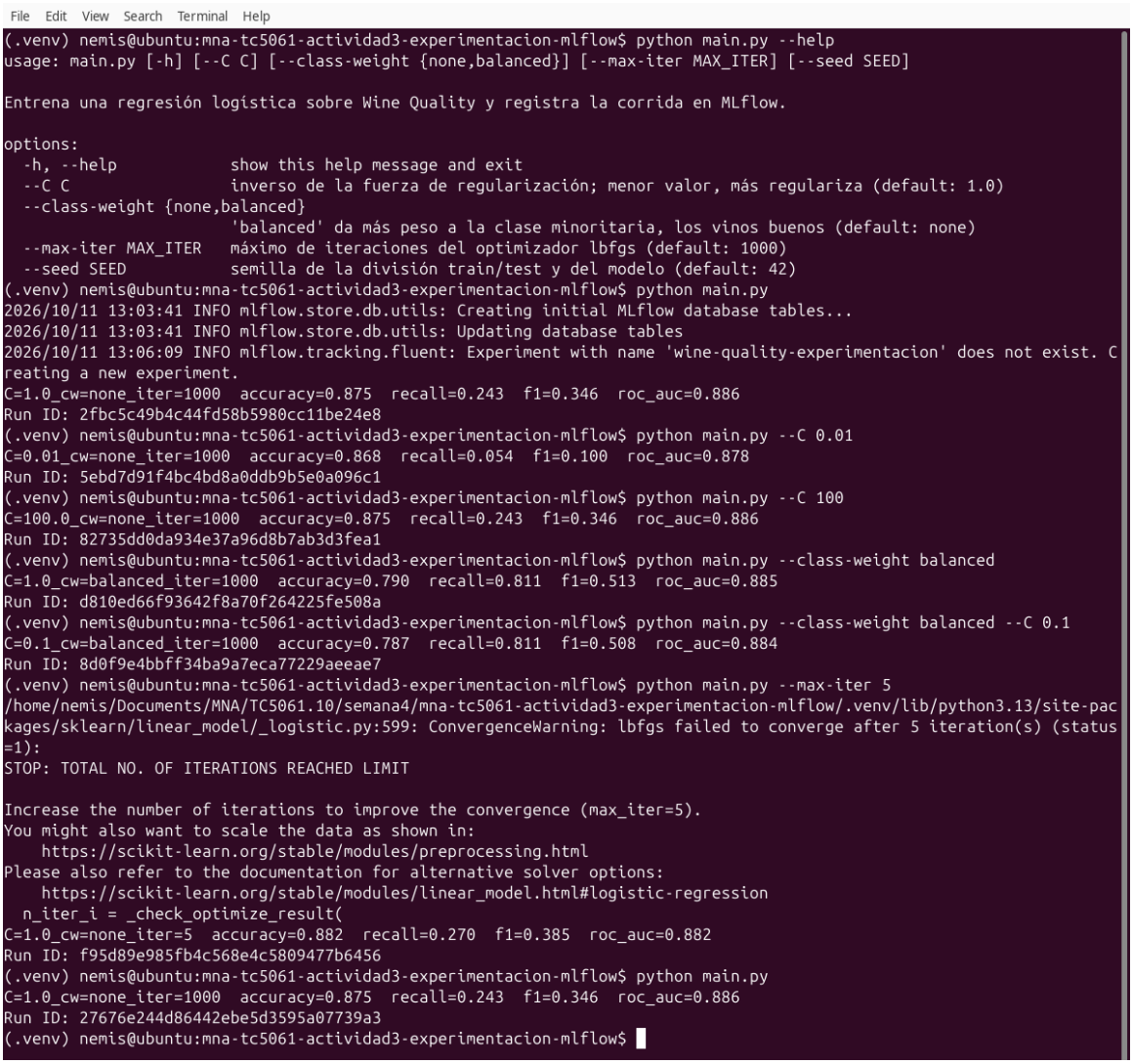


Figura 1: `python main.py --help` con la ayuda de cada argumento, la corrida sin argumentos y las siete corridas. La primera y la última imprimen las mismas métricas.

3. Instrumentación con MLflow Tracking

Todo el registro ocurre dentro de un `mlflow.start_run()` (Código 2, Figuras 2 y 3).

Requisito	Implementación
Hiperparámetros con `log_param`	`C`, `class_weight`, `max_iter` y `seed`
Métricas con `log_metric`	accuracy, recall, F1 y ROC-AUC
Artefactos con `log_artifact`	`matriz_confusion.png` y `classification_report.txt`
Modelo con `mlflow.sklearn.log_model`	`Pipeline` de `StandardScaler` y `LogisticRegression`

Código 2: `main.py`: registro de la corrida dentro de `mlflow.start_run()`.

```
    mlflow.set_experiment(EXPERIMENT_NAME)
    with mlflow.start_run(run_name=nombre) as run:
        for parametro, valor in vars(args).items():
            mlflow.log_param(parametro, valor)
        for metrica, valor in metricas.items():
            mlflow.log_metric(metrica, valor)
        with tempfile.TemporaryDirectory() as carpeta:
            for ruta in guardar_artefactos(y_test, pred, Path(carpeta)):
                mlflow.log_artifact(str(ruta))
        mlflow.sklearn.log_model(modelo, name="model", input_example=X_train.head(3))
```

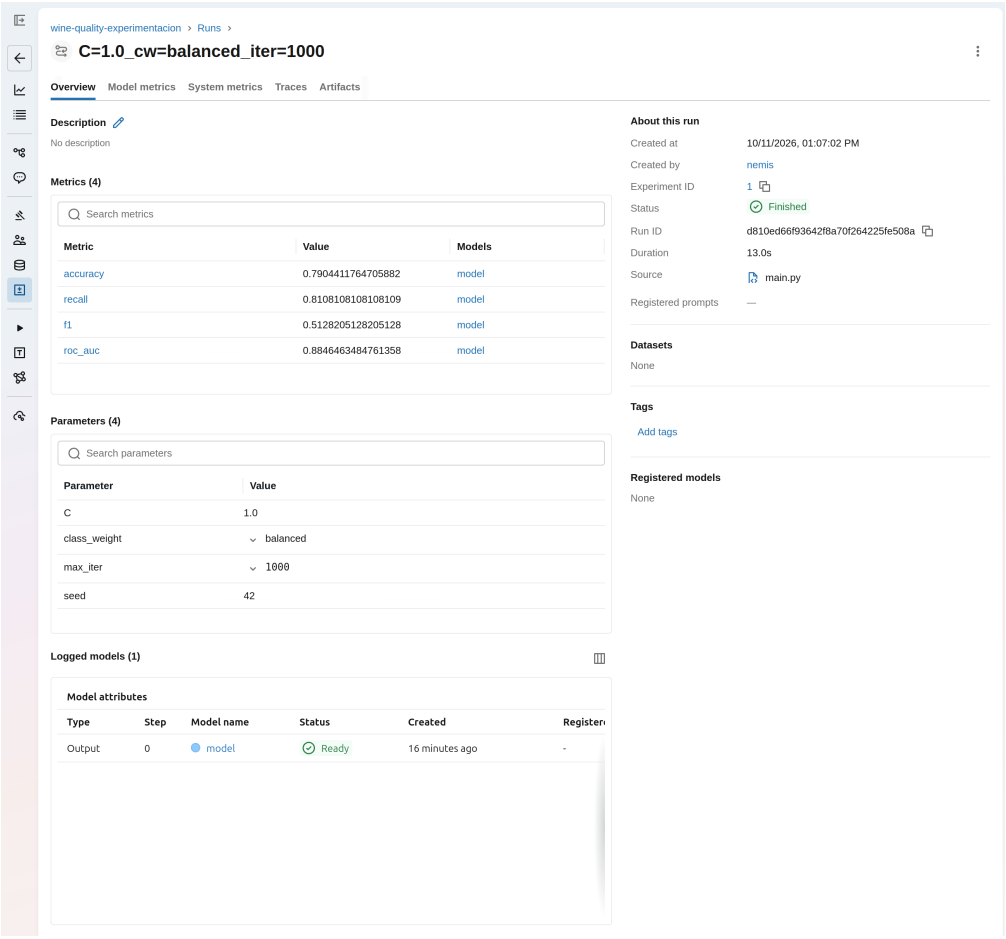


Figura 2: Detalle de la corrida 4 en MLflow: métricas (`log_metric`), parámetros (`log_param`) y el modelo registrado (`log_model`).

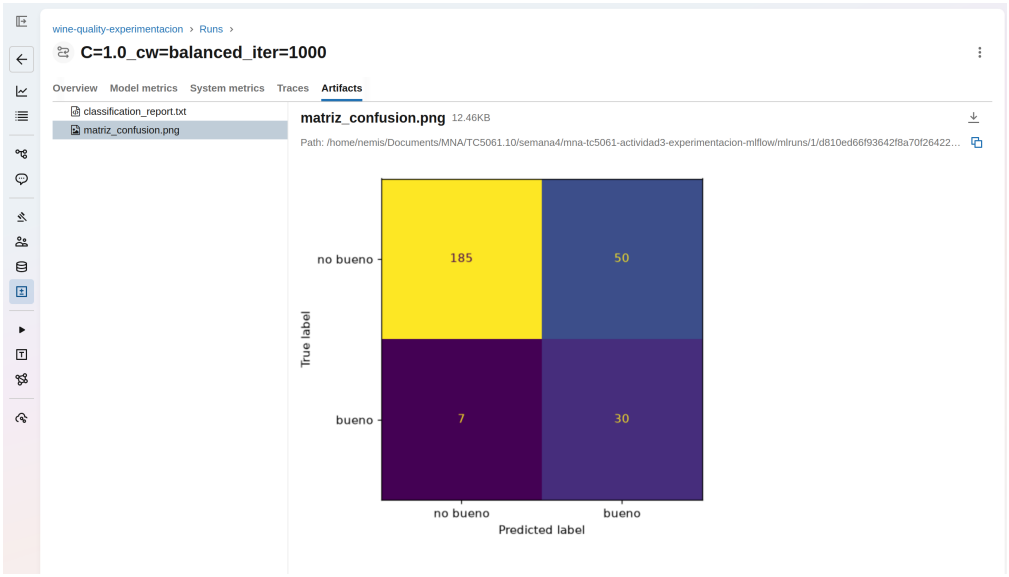


Figura 3: Pestaña *Artifacts* de la misma corrida: la matriz de confusión y el classification report (`log_artifact`).

4. Ejecución y comparación de corridas

Siete corridas en el experimento `wine-quality-experimentacion`, contrastadas en MLflow (Figura 4).

#	Argumentos	accuracy	recall	F1	ROC-AUC
1	por defecto (C=1, sin pesos, 1000 iteraciones)	0.875	0.243	0.346	0.886
2	`--C 0.01`	0.868	0.054	0.100	0.878
3	`--C 100`	0.875	0.243	0.346	0.886
4	`--class-weight balanced`	0.790	**0.811**	**0.513**	0.885
5	`--class-weight balanced --C 0.1`	0.787	0.811	0.508	0.884
6	`--max-iter 5`	0.882	0.270	0.385	0.882
7	por defecto, repetida	0.875	0.243	0.346	0.886

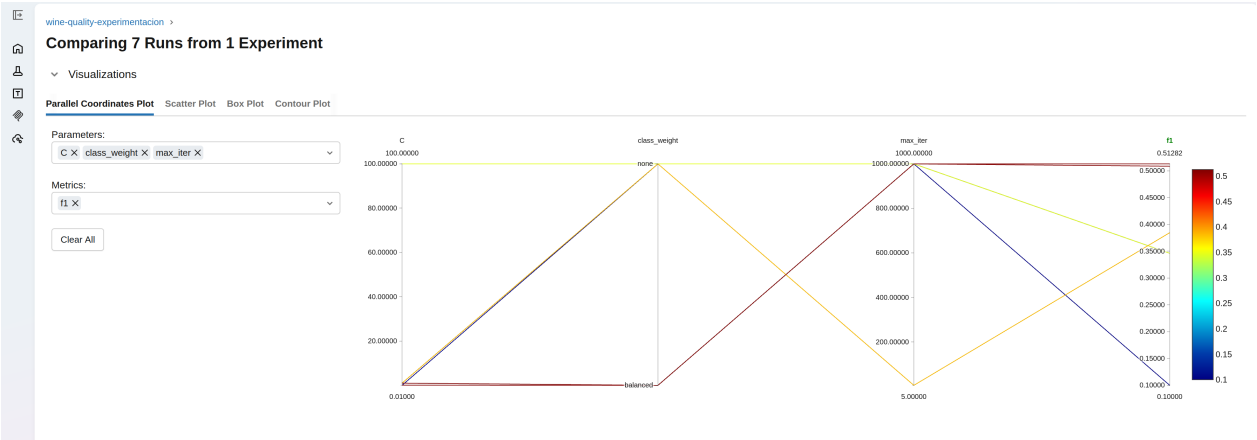


Figura 4: Vista de comparación de MLflow con las siete corridas del experimento `wine-quality-experimentacion`, en coordenadas paralelas coloreadas por F1.

5. Fijación de la semilla y dependencias

La semilla controla la partición estratificada de `train_test_split`. También se pasa a `LogisticRegression`, pero con el solver lbfgs ese parámetro no interviene. La corrida 7 repite la 1 y sus métricas coinciden sin redondear (Tabla 1, exportada de MLflow con sus IDs). Entorno: Ubuntu 24.04, Python 3.13.15 y las versiones fijadas en `requirements.txt` (Código 3), entre ellas mlflow 3.16.1 y scikit-learn 1.9.1.

Código 3: `requirements.txt` con las versiones utilizadas.

```
# Python 3.13. Versiones fijadas para que dos ejecuciones con los mismos argumentos coincidan
mlflow==3.16.1
scikit-learn==1.9.1
pandas==3.0.6
numpy==2.5.3
matplotlib==3.11.2

# Solo para abrir el notebook de exploración
jupyter==1.1.1
```

Tabla 1: Métricas sin redondear de las corridas 1 y 7, exportadas de MLflow con `mlflow.search_runs` (las siete corridas están en `reports/corridas_mlflow.csv`).

	Corrida 1	**Corrida 7**
Run ID	`2fbc5c49b4c44fd58b5980cc11be24e8`	`27676e244d86442ebe5d3595a07739a3`
Argumentos	por defecto	por defecto
accuracy	0.875	0.875
recall	0.24324324324324326	0.24324324324324326
F1	0.34615384615384615	0.34615384615384615
ROC-AUC	0.88602645198389884	0.88602645198389884

6. Análisis de resultados

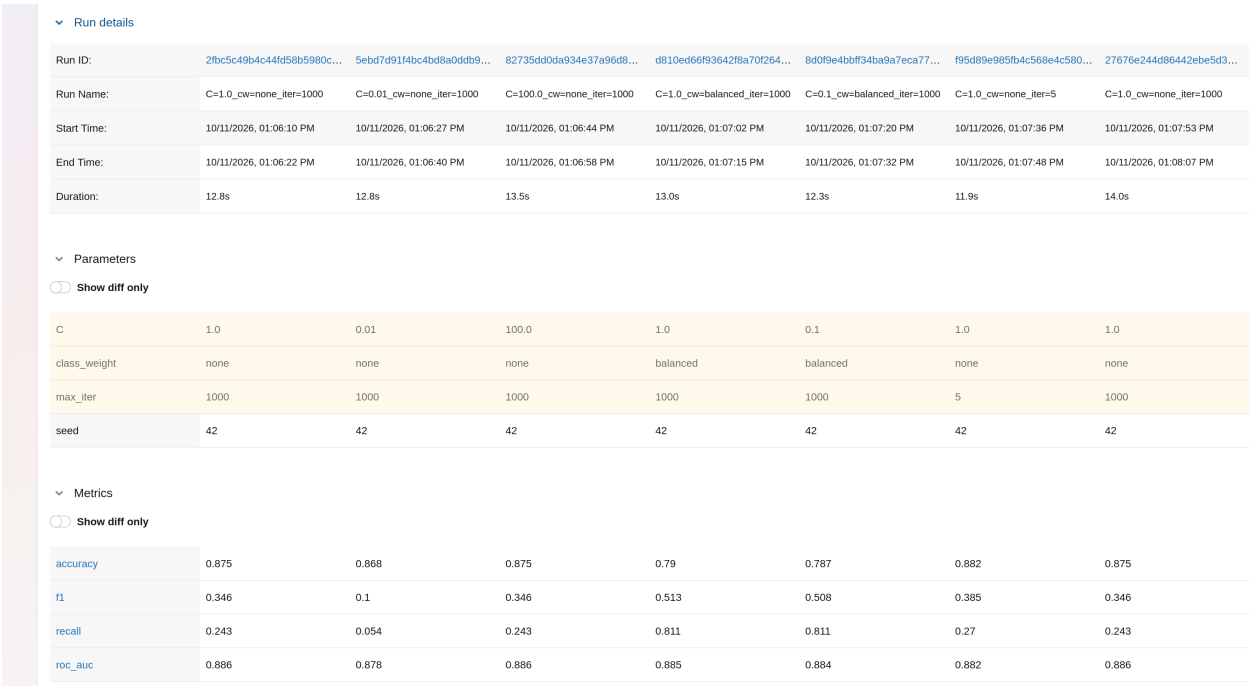
Mejor configuración. Entre las configuraciones probadas y en esta partición, la mejor por F1 es la corrida 4, `--class-weight balanced` con C=1 (Figura 5): el F1 sube de 0.346 a 0.513. El recall pasa de 0.243 a 0.811 (30 de 37 vinos buenos), pero la precisión positiva es de 37.5 %: de los 80 vinos que marca como buenos, 50 no lo son. Ese es el costo del recall y la razón de que el accuracy baje a 0.790. El ROC-AUC parecido (0.886 contra 0.885) indica una capacidad de discriminación similar. `class_weight` cambia el ajuste del modelo al dar más peso a los vinos buenos, y el umbral de decisión sigue en 0.5.

Hiperparámetro más influyente. Depende de cómo se mida. `class_weight=balanced` produjo la mayor mejora de F1 (+0.167). El mayor cambio absoluto lo produjo `C=0.01` (−0.246, F1 de 0.100), porque la regularización fuerte casi elimina las predicciones positivas. Entre C=1 y C=100 se mantuvieron las métricas y las clases predichas, aunque cambiaron las probabilidades. lbfgs convergió en 12 iteraciones (baseline), 7 (C=0.01) y 13 (C=100 y balanceadas), así que `max_iter=1000` no limitó el ajuste. Con `max_iter=5` no hubo convergencia y su F1 de 0.385 corresponde a un modelo incompleto, no a una configuración recomendable. Estas conclusiones valen solo para las siete configuraciones probadas.

Limitaciones. El conjunto de prueba tiene 37 vinos buenos, así que esta evaluación no permite establecer una

superioridad estable entre configuraciones cercanas: entre las dos balanceadas solo cambia un falso positivo (50 frente a 51). Además, la configuración se eligió con el mismo conjunto de prueba, lo que hace optimista la estimación. Validación cruzada, otros umbrales y otros modelos quedan como mejoras.

Más allá de qué configuración ganó, lo que me deja esta actividad es otra forma de trabajar. En la semana 3 los resultados dependían de correr las celdas en el orden correcto y de acordarme de qué había cambiado. Ahora cada corrida queda guardada con sus parámetros, sus métricas y su modelo, y cualquiera puede repetirla con un solo comando. Eso es lo que permite comparar con confianza y no solo con intuición.



Run details

Run ID:	2fbc5c49b4c44fd58b5980c…	5ebd7d91f4bc4bd8a0ddb9…	82735dd0da934e37a96d8…	d810ed66f93642f8a70f264…	8d0f9e4bbff34ba9a7eca77…	f95d89e985fb4c568e4c580…	27676e244d86442ebe5d3…
Run Name:	C=1.0_cw=none_iter=1000	C=0.01_cw=none_iter=1000	C=100.0_cw=none_iter=1000	C=1.0_cw=balanced_iter=1000	C=0.1_cw=balanced_iter=1000	C=1.0_cw=none_iter=5	C=1.0_cw=none_iter=1000
Start Time:	10/11/2026, 01:06:10 PM	10/11/2026, 01:06:27 PM	10/11/2026, 01:06:44 PM	10/11/2026, 01:07:02 PM	10/11/2026, 01:07:20 PM	10/11/2026, 01:07:36 PM	10/11/2026, 01:07:53 PM
End Time:	10/11/2026, 01:06:22 PM	10/11/2026, 01:06:40 PM	10/11/2026, 01:06:58 PM	10/11/2026, 01:07:15 PM	10/11/2026, 01:07:32 PM	10/11/2026, 01:07:48 PM	10/11/2026, 01:08:07 PM
Duration:	12.8s	12.8s	13.5s	13.0s	12.3s	11.9s	14.0s

Parameters

Show diff only

C	1.0	0.01	100.0	1.0	0.1	1.0	1.0
class_weight	none	none	none	balanced	balanced	none	none
max_iter	1000	1000	1000	1000	1000	5	1000
seed	42	42	42	42	42	42	42

Metrics

Show diff only

accuracy	0.875	0.868	0.875	0.79	0.787	0.882	0.875
f1	0.346	0.1	0.346	0.513	0.508	0.385	0.346
recall	0.243	0.054	0.243	0.811	0.811	0.27	0.243
roc_auc	0.886	0.878	0.886	0.885	0.884	0.882	0.886

Figura 5: Tabla comparativa de MLflow con los parámetros y las métricas de las siete corridas.

Declaración de uso de herramientas de inteligencia artificial

En esta entrega se utilizó Claude Code de Anthropic, con el modelo Claude Opus 5.5, para la generación parcial de código, orientada a mejorar el código propuesto y a plantear alternativas basadas en decisiones de optimización.

Anthropic. (2026). *Claude Code (Claude Opus 5.5)* [Modelo de lenguaje grande], utilizado para generación parcial de código, mejora del código propuesto y alternativas de optimización. `https://claude.com/claude-code`

Referencias

Cortez, P., Cerdeira, A., Almeida, F., Matos, T. y Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems, 47*(4), 547–553. https://doi.org/10.1016/j.dss.2009.05.016

DrivenData. (s. f.). *Cookiecutter Data Science*. Recuperado el 11 de octubre de 2026, de https://cookiecutter-data-science.drivendata.org/

MLflow. (s. f.). *MLflow Tracking* [Documentación]. Recuperado el 11 de octubre de 2026, de https://mlflow.org/docs/latest/ml/tracking/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.